ARCHIENGINE PRO - PROJECT FILE LIST
==========

Copy each section below into separate files using your text editor
==========

FILE 1: package.json

```
{
  "name": "archiengine-pro",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "postinstall": "prisma generate"
  },
  "dependencies": {
    "@auth/prisma-adapter": "^1.0.0",
    "@prisma/client": "^5.7.0",
    "@stripe/react-stripe-js": "^2.4.0",
    "@stripe/stripe-js": "^2.2.0",
    "bcryptjs": "^2.4.3",
    "clsx": "^2.0.0",
    "date-fns": "^3.0.0",
    "framer-motion": "^10.16.16",
    "lucide-react": "^0.294.0",
    "next": "14.1.0",
    "next-auth": "^4.24.5",
    "openai": "^4.24.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "recharts": "^2.10.3",
    "stripe": "^14.9.0",
    "tailwind-merge": "^2.2.0",
    "zod": "^3.22.4"
  },
  "devDependencies": {
    "@types/bcryptjs": "^2.4.6",
    "@types/node": "^20.10.0",
    "@types/react": "^18.2.0",
    "autoprefixer": "^10.4.0",
    "postcss": "^8.4.0",
    "prisma": "^5.7.0",
    "tailwindcss": "^3.3.0",
    "typescript": "^5.3.0"
  }
}
```

==========

FILE 2: next.config.js

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  images: {
    unoptimized: true,
  },
  trailingSlash: true,
};

module.exports = nextConfig;
```

==========

FILE 3: tsconfig.json

```
{
  "compilerOptions": {
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [{ "name": "next" }],
    "paths": { "@/*": ["./*"] }
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules"]
}
```

==========

FILE 4: tailwind.config.ts

```
import type { Config } from 'tailwindcss';
```

```
const config: Config = {
  content: [
    './app/**/*.{js,ts,jsx,tsx,mdx}',
    './components/**/*.{js,ts,jsx,tsx,mdx}',
  ],
  theme: {
    extend: {
      colors: {
        border: "hsl(217 33% 17%)",
        input: "hsl(217 33% 17%)",
        ring: "hsl(217 91% 60%)",
        background: "hsl(222 47% 4%)",
        foreground: "hsl(210 40% 98%)",
        primary: {
          DEFAULT: "hsl(217 91% 60%)",
          foreground: "hsl(222 47% 4%)",
        },
        secondary: {
          DEFAULT: "hsl(217 33% 17%)",
          foreground: "hsl(210 40% 98%)",
        },
        card: {
          DEFAULT: "hsl(222 47% 6%)",
          foreground: "hsl(210 40% 98%)",
        },
      },
      borderRadius: {
        lg: "0.75rem",
        md: "calc(0.75rem - 2px)",
        sm: "calc(0.75rem - 4px)",
      },
    },
  },
  plugins: [],
};
export default config;
```

==========

FILE 5: postcss.config.js

```
module.exports = {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
};
```

==========

FILE 6: .env.local

```
# Database (Supabase)
DATABASE_URL="postgresql://postgres.[password]@db.[project].supabase.co:5432/postgres"
```

```
# NextAuth
NEXTAUTH_URL="http://localhost:3000"
NEXTAUTH_SECRET="your-super-secret-key-min-32-characters"

# Stripe
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY="pk_test_..."
STRIPE_SECRET_KEY="sk_test_..."
STRIPE_WEBHOOK_SECRET="whsec_..."
STRIPE_PRICE_ID_PRO="price_..."
STRIPE_PRICE_ID_ENTERPRISE="price_..."

# OpenAI
OPENAI_API_KEY="sk-..."

# Admin
ADMIN_EMAIL="admin@yourdomain.com"
```

==========

FILE 7: .env.local.example

```
# Database (Supabase)
DATABASE_URL="postgresql://postgres.[password]@db.[project].supabase.co:5432/postgres"

# NextAuth
NEXTAUTH_URL="http://localhost:3000"
NEXTAUTH_SECRET="generate-a-32-char-secret-here"

# Stripe
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY="pk_test_..."
STRIPE_SECRET_KEY="sk_test_..."
STRIPE_WEBHOOK_SECRET="whsec_..."
STRIPE_PRICE_ID_PRO="price_..."
STRIPE_PRICE_ID_ENTERPRISE="price_..."

# OpenAI
OPENAI_API_KEY="sk-..."

# Admin
ADMIN_EMAIL="admin@yourdomain.com"
```

==========

FILE 8: .gitignore

```
# Dependencies
/node_modules
/.pnp
.pnp.js

# Testing
/coverage

# Next.js
/.next/
/out/
```

```
# Production
/build

# Misc
.DS_Store
*.pem

# Debug
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# Local env files
.env*.local
.env

# Vercel
.vercel

# TypeScript
*.tsbuildinfo
next-env.d.ts

# IDE
.vscode/
.idea/
```

==========

FILE 9: README.md

🏗️ ArchiEngine Pro

AI-Powered Architectural Design Platform

Transform your architectural vision into photorealistic designs in seconds.

Features
- 🤖 AI Design Generation
- 💰 Subscription System (Free, Pro, Enterprise)
- 🔐 Secure Authentication
- 📊 Admin Dashboard
- 💳 Stripe Payments

Quick Start
1. Clone repository
2. Run `npm install`
3. Copy `.env.local.example` to `.env.local`
4. Add your API keys
5. Run `npm run dev`

Tech Stack
- Next.js 14
- TypeScript
- Tailwind CSS
- Prisma + PostgreSQL
- NextAuth.js
- Stripe
- OpenAI DALL-E

License
MIT

==========

FILE 10: prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Account {
  id                String  @id @default(cuid())
  userId            String
  type              String
  provider          String
  providerAccountId String
  refresh_token     String? @db.Text
  access_token      String? @db.Text
  expires_at        Int?
  token_type        String?
  scope             String?
  id_token          String? @db.Text
  session_state     String?
  user              User    @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@unique([provider, providerAccountId])
}

model Session {
  id           String   @id @default(cuid())
  sessionToken String   @unique
  userId       String
  expires      DateTime
  user         User     @relation(fields: [userId], references: [id], onDelete: Cascade)
}
```

```
model User {
  id            String        @id @default(cuid())
  name          String?
  email         String        @unique
  emailVerified DateTime?
  image         String?
  password      String?
  role          UserRole      @default(USER)
  accounts      Account[]
  sessions      Session[]
  projects      Project[]
  subscription  Subscription?
  payments      Payment[]
  aiUsage       AIUsage[]
  createdAt     DateTime      @default(now())
  updatedAt     DateTime      @updatedAt
}

enum UserRole {
  USER
  ADMIN
  PREMIUM
}

model VerificationToken {
  identifier String
  token      String   @unique
  expires    DateTime
  @@unique([identifier, token])
}

model Project {
  id          String       @id @default(cuid())
  userId      String
  user        User         @relation(fields: [userId], references: [id], onDelete: Cascade)
  title       String
  description String?
  type        ProjectType   @default(RESIDENTIAL)
  status      ProjectStatus @default(DRAFT)
  aiDesigns   AIDesign[]
  createdAt   DateTime      @default(now())
  updatedAt   DateTime      @updatedAt
}

enum ProjectType {
  RESIDENTIAL
  COMMERCIAL
  INDUSTRIAL
  LANDSCAPE
  INTERIOR
  URBAN
}
```

```
enum ProjectStatus {
  DRAFT
  IN_PROGRESS
  COMPLETED
  ARCHIVED
}

model AIDesign {
  id        String   @id @default(cuid())
  projectId String
  project   Project  @relation(fields: [projectId], references: [id], onDelete: Cascade)
  prompt    String   @db.Text
  imageUrl  String?
  settings  Json?
  createdAt DateTime @default(now())
}

model Subscription {
  id                  String              @id @default(cuid())
  userId              String              @unique
  user                User                @relation(fields: [userId], references: [id], onDelete: Cascade)
  status              SubscriptionStatus @default(INCOMPLETE)
  plan                PlanType            @default(FREE)
  priceId             String?
  subscriptionId      String?             @unique
  currentPeriodStart DateTime?
  currentPeriodEnd   DateTime?
  cancelAtPeriodEnd  Boolean             @default(false)
  createdAt           DateTime            @default(now())
  updatedAt           DateTime            @updatedAt
}

enum SubscriptionStatus {
  INCOMPLETE
  INCOMPLETE_EXPIRED
  TRIALING
  ACTIVE
  PAST_DUE
  CANCELED
  UNPAID
  PAUSED
}

enum PlanType {
  FREE
  PRO
  ENTERPRISE
}
```

```
model Payment {
  id            String       @id @default(cuid())
  userId        String
  user          User        @relation(fields: [userId], references: [id], onDelete: Cascade)
  amount        Decimal      @db.Decimal(10, 2)
  currency      String       @default("USD")
  status        PaymentStatus @default(PENDING)
  paymentIntentId String?      @unique
  description    String?
  createdAt      DateTime      @default(now())
}

enum PaymentStatus {
  PENDING
  COMPLETED
  FAILED
  REFUNDED
  DISPUTED
}

model AIUsage {
  id        String   @id @default(cuid())
  userId    String
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  model     String
  tokensUsed Int?
  cost      Decimal  @db.Decimal(10, 6)
  prompt    String   @db.Text
  createdAt  DateTime @default(now())
}
```

==========

FILE 11: lib/prisma.ts

```
import { PrismaClient } from '@prisma/client';

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};

export const prisma = globalForPrisma.prisma ?? new PrismaClient({
  log: process.env.NODE_ENV === 'development' ? ['query', 'error', 'warn'] : ['error'],
});

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
```

==========

FILE 12: lib/auth.ts

```
import { NextAuthOptions } from 'next-auth';
import { PrismaAdapter } from '@auth/prisma-adapter';
import CredentialsProvider from 'next-auth/providers/credentials';
import { prisma } from './prisma';
import bcrypt from 'bcryptjs';
import { z } from 'zod';
```

```
const credentialsSchema = z.object({
  email: z.string().email(),
  password: z.string().min(12),
});
```

```
export const authOptions: NextAuthOptions = {
  adapter: PrismaAdapter(prisma) as any,
  session: {
    strategy: 'jwt',
    maxAge: 30 * 24 * 60 * 60,
  },
  pages: {
    signIn: '/auth/login/',
    error: '/auth/error/',
  },
  providers: [
    CredentialsProvider({
      name: 'credentials',
      credentials: {
        email: { label: 'Email', type: 'email' },
        password: { label: 'Password', type: 'password' },
      },
      async authorize(credentials) {
        try {
          const { email, password } = credentialsSchema.parse(credentials);
          const user = await prisma.user.findUnique({ where: { email } });
          if (!user || !user.password) return null;
          const isValid = await bcrypt.compare(password, user.password);
          if (!isValid) return null;
          return {
            id: user.id,
            email: user.email,
            name: user.name,
            role: user.role,
            image: user.image,
          };
        } catch (error) {
          return null;
        }
      },
    }),
  ],
  callbacks: {
    async jwt({ token, user }) {
      if (user) {
        token.role = user.role;
        token.id = user.id;
      }
      return token;
    },
    async session({ session, token }) {
      if (token) {
        session.user.id = token.id;
        session.user.role = token.role;
      }
      return session;
    },
  },
};
```

==========

FILE 13: lib/password.ts

```
import bcrypt from 'bcryptjs';
import { z } from 'zod';

export const passwordSchema = z
  .string()
  .min(12, 'Password must be at least 12 characters')
  .regex(/[A-Z]/, 'Must contain uppercase')
  .regex(/[a-z]/, 'Must contain lowercase')
  .regex(/[0-9]/, 'Must contain number')
  .regex(/[^A-Za-z0-9]/, 'Must contain special character');

export async function hashPassword(password: string): Promise<string> {
  return bcrypt.hash(password, 14);
}
```

==========

FILE 14: lib/stripe.ts

```
import Stripe from 'stripe';

export const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: '2023-10-16',
});

export const STRIPE_PUBLISHABLE_KEY = process.env.NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY!;
```

```
export const PRICING_PLANS = {
  FREE: {
    id: 'free',
    name: 'Free',
    description: 'Perfect for trying out AI architectural design',
    price: 0,
    priceId: null,
    features: [
      '3 AI-generated designs per month',
      'Basic architectural styles',
      'Standard resolution renders',
      'Community support',
      'Project storage up to 5 projects',
    ],
    cta: 'Get Started Free',
    popular: false,
  },
  PRO: {
    id: 'pro',
    name: 'Professional',
    description: 'For serious architects and designers',
    price: 29.99,
    priceId: process.env.STRIPE_PRICE_ID_PRO!,
    features: [
      'Unlimited AI-generated designs',
      'All architectural styles',
      'High-resolution renders',
      'Priority processing',
      'Export to CAD formats',
      'Commercial usage rights',
      'Email support',
    ],
    cta: 'Start Pro Trial',
    popular: true,
  },
  ENTERPRISE: {
    id: 'enterprise',
    name: 'Enterprise',
    description: 'For firms and large teams',
    price: 99.99,
    priceId: process.env.STRIPE_PRICE_ID_ENTERPRISE!,
    features: [
      'Everything in Professional',
      'Ultra HD renders',
      'API access',
      'Custom AI training',
      'White-label options',
      'Dedicated support',
    ],
    cta: 'Contact Sales',
    popular: false,
  },
};

==================================================================
```

FILE 15: lib/ai.ts

```
import OpenAI from 'openai';
import { prisma } from './prisma';
import { PlanType } from '@prisma/client';

const openai = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY,
});

const RATE_LIMITS = { FREE: 3, PRO: 1000, ENTERPRISE: 10000 };
const MODELS = { FREE: 'dall-e-2', PRO: 'dall-e-3', ENTERPRISE: 'dall-e-3' };
const SIZES = { FREE: '512x512', PRO: '1024x1024', ENTERPRISE: '1792x1024' };

interface GenerateParams {
  prompt: string;
  userId: string;
  plan: PlanType;
  projectId: string;
  style?: string;
  setting?: string;
  timeOfDay?: string;
}

export async function generateArchitecturalDesign({
  prompt, userId, plan, projectId, style = 'modern', setting = 'urban', timeOfDay = 'daylight'
}: GenerateParams) {

  if (plan === 'FREE') {
    const usageCount = await prisma.aiDesign.count({
      where: {
        project: { userId },
        createdAt: { gte: new Date(new Date().setDate(new Date().getDate() - 30)) },
      },
    });
    if (usageCount >= RATE_LIMITS.FREE) {
      throw new Error('Free tier limit reached. Upgrade to Pro for unlimited designs.');
    }
  }

  const enhancedPrompt = `Professional architectural visualization: ${prompt}. Style: ${style} architecture.
Setting: ${setting} environment. Lighting: ${timeOfDay}. Photorealistic 3D render, high-quality materials,
professional composition, 8K quality.`;

  const response = await openai.images.generate({
    model: MODELS[plan],
    prompt: enhancedPrompt,
    size: SIZES[plan] as any,
    n: 1,
  });

  const cost = plan === 'FREE' ? 0.016 : plan === 'PRO' ? 0.08 : 0.12;
```

```
  const design = await prisma.aiDesign.create({
    data: {
      projectId,
      prompt: enhancedPrompt,
      imageUrl: response.data[0].url,
      settings: { style, setting, timeOfDay, model: MODELS[plan] },
    },
  });

  await prisma.aiUsage.create({
    data: { userId, model: MODELS[plan], cost, prompt: prompt.substring(0, 500) },
  });

  return { id: design.id, url: response.data[0].url, prompt: enhancedPrompt };
}
```

==========

FILE 16: lib/utils.ts

```
import { type ClassValue, clsx } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export function formatCurrency(amount: number): string {
  return new Intl.NumberFormat('en-US', {
    style: 'currency',
    currency: 'USD',
  }).format(amount);
}
```

==========

FILE 17: types/next-auth.d.ts

```
import NextAuth from 'next-auth';

declare module 'next-auth' {
  interface Session {
    user: {
      id: string;
      name?: string | null;
      email?: string | null;
      image?: string | null;
      role: 'USER' | 'ADMIN' | 'PREMIUM';
    };
  }

  interface User {
    role: 'USER' | 'ADMIN' | 'PREMIUM';
  }
}
```

```
declare module 'next-auth/jwt' {
  interface JWT {
    role?: 'USER' | 'ADMIN' | 'PREMIUM';
    id?: string;
  }
}
```

==========

FILE 18: components/providers.tsx

```
'use client';

import { SessionProvider } from 'next-auth/react';

export function Providers({ children }: { children: React.ReactNode }) {
  return <SessionProvider>{children}</SessionProvider>;
}
```

==========

FILE 19: components/pricing-card.tsx

```
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';
import { loadStripe } from '@stripe/stripe-js';
import { STRIPE_PUBLISHABLE_KEY } from '@/lib/stripe';
import { Check, Sparkles } from 'lucide-react';

const stripePromise = loadStripe(STRIPE_PUBLISHABLE_KEY);

interface PricingCardProps {
  plan: any;
  session: any;
  popular?: boolean;
}

export function PricingCard({ plan, session, popular }: PricingCardProps) {
  const [isLoading, setIsLoading] = useState(false);
  const router = useRouter();

  const handleSubscribe = async () => {
    if (!session) {
      router.push('/auth/login/?callbackUrl=/pricing/');
      return;
    }

    if (plan.price === 0) {
      router.push('/dashboard/');
      return;
    }
```

```
    setIsLoading(true);
    try {
      const response = await fetch('/api/stripe/checkout/', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ priceId: plan.priceId, planType: plan.id.toUpperCase() }),
      });
      const { url } = await response.json();
      window.location.href = url;
    } catch (error) {
      alert('Something went wrong. Please try again.');
    } finally {
      setIsLoading(false);
    }
  };

  return (
    <div className={`relative rounded-2xl p-8 ${popular ? 'bg-gradient-to-b from-blue-600 to-blue-800 border-2
border-blue-400' : 'bg-slate-800/50 border border-slate-700'}`}>
      {popular && (
        <div className="absolute -top-4 left-1/2 -translate-x-1/2">
          <span className="bg-gradient-to-r from-yellow-400 to-orange-500 text-slate-900 px-4 py-1 rounded-full
text-sm font-bold flex items-center gap-1">
            <Sparkles className="h-4 w-4" /> Most Popular
          </span>
        </div>
      )}

      <h3 className="text-2xl font-bold mb-2 text-white">{plan.name}</h3>
      <p className={`mb-6 ${popular ? 'text-blue-100' : 'text-slate-400'}`}>{plan.description}</p>

      <div className="mb-6">
        <span className="text-4xl font-bold text-white">${plan.price}</span>
        {plan.price > 0 && <span className={popular ? 'text-blue-200' : 'text-slate-400'}>/month</span>}
      </div>

      <ul className="space-y-4 mb-8">
        {plan.features.map((feature: string) => (
          <li key={feature} className="flex items-start gap-3">
            <Check className={`h-5 w-5 mt-0.5 ${popular ? 'text-blue-200' : 'text-emerald-400'}`} />
            <span className={popular ? 'text-blue-100' : 'text-slate-300'}>{feature}</span>
          </li>
        ))}
      </ul>

      <button
        onClick={handleSubscribe}
        disabled={isLoading}
        className={`w-full py-3 rounded-xl font-semibold transition ${popular ? 'bg-white text-blue-600 hover:bg-
blue-50' : 'bg-blue-600 text-white hover:bg-blue-700'} disabled:opacity-50`}
      >
        {isLoading ? 'Loading...' : plan.cta}
      </button>
    </div>
  );
}
```

==========

FILE 20: components/ui/card.tsx

```
import * as React from "react";
import { cn } from "@/lib/utils";

const Card = React.forwardRef<HTMLDivElement, React.HTMLAttributes<HTMLDivElement>>(
  ({ className, ...props }, ref) => (
    <div ref={ref} className={cn("rounded-xl border bg-card text-card-foreground shadow", className)} {...props}
/>
  )
);
Card.displayName = "Card";

const CardHeader = React.forwardRef<HTMLDivElement, React.HTMLAttributes<HTMLDivElement>>(
  ({ className, ...props }, ref) => (
    <div ref={ref} className={cn("flex flex-col space-y-1.5 p-6", className)} {...props} />
  )
);
CardHeader.displayName = "CardHeader";

const CardTitle = React.forwardRef<HTMLParagraphElement, React.HTMLAttributes<HTMLHeadingElement>>(
  ({ className, ...props }, ref) => (
    <h3 ref={ref} className={cn("font-semibold leading-none tracking-tight", className)} {...props} />
  )
);
CardTitle.displayName = "CardTitle";

const CardContent = React.forwardRef<HTMLDivElement, React.HTMLAttributes<HTMLDivElement>>(
  ({ className, ...props }, ref) => (
    <div ref={ref} className={cn("p-6 pt-0", className)} {...props} />
  )
);
CardContent.displayName = "CardContent";

export { Card, CardHeader, CardTitle, CardContent };
```

==========

FILE 21: app/globals.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

```
@layer base {
  :root {
    --background: 222 47% 4%;
    --foreground: 210 40% 98%;
    --card: 222 47% 6%;
    --card-foreground: 210 40% 98%;
    --primary: 217 91% 60%;
    --primary-foreground: 222 47% 4%;
    --secondary: 217 33% 17%;
    --secondary-foreground: 210 40% 98%;
    --muted: 217 33% 17%;
    --muted-foreground: 215 20% 65%;
    --border: 217 33% 17%;
    --input: 217 33% 17%;
    --ring: 217 91% 60%;
    --radius: 0.75rem;
  }
}

@layer base {
  * {
    @apply border-border;
  }
  body {
    @apply bg-background text-foreground;
  }
}

::-webkit-scrollbar {
  width: 8px;
}

::-webkit-scrollbar-track {
  background: #0f172a;
}

::-webkit-scrollbar-thumb {
  background: #334155;
  border-radius: 4px;
}

==================================================================

FILE 22: app/layout.tsx
----------------------
import type { Metadata } from 'next';
import { Inter } from 'next/font/google';
import './globals.css';
import { Providers } from '@/components/providers';

const inter = Inter({ subsets: ['latin'] });

export const metadata: Metadata = {
  title: 'ArchiEngine Pro - AI Architectural Design',
  description: 'Transform your architectural vision into photorealistic designs with AI',
};
```

```
export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body className={inter.className}>
        <Providers>{children}</Providers>
      </body>
    </html>
  );
}

==================================================================

FILE 23: app/page.tsx
--------------------
import Link from 'next/link';
import { getServerSession } from 'next-auth';
import { authOptions } from '@/lib/auth';
import { redirect } from 'next/navigation';
import { ArrowRight, Sparkles, Shield, Zap, Building2 } from 'lucide-react';

export default async function LandingPage() {
  const session = await getServerSession(authOptions);
  if (session?.user) redirect('/dashboard/');

  return (
    <div className="min-h-screen bg-slate-950 text-white">
      <nav className="fixed top-0 w-full z-50 bg-slate-950/80 backdrop-blur-md border-b border-slate-800">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
          <div className="flex justify-between items-center h-16">
            <div className="flex items-center gap-2">
              <Building2 className="h-8 w-8 text-blue-500" />
              <span className="text-xl font-bold bg-gradient-to-r from-blue-400 to-purple-500 bg-clip-text text-
transparent">
                ArchiEngine
              </span>
            </div>
            <div className="flex items-center gap-4">
              <Link href="/auth/login/" className="text-slate-300 hover:text-white">Sign In</Link>
              <Link href="/auth/register/" className="bg-blue-600 hover:bg-blue-700 px-4 py-2 rounded-lg">Get
Started</Link>
            </div>
          </div>
        </div>
      </nav>
```

```
<section className="pt-32 pb-20 px-4">
  <div className="max-w-7xl mx-auto text-center">
    <div className="inline-flex items-center gap-2 px-4 py-2 rounded-full bg-blue-500/10 border border-blue-500/20 mb-8">
      <Sparkles className="h-4 w-4 text-blue-400" />
      <span className="text-sm text-blue-300">Powered by Advanced AI</span>
    </div>

    <h1 className="text-5xl md:text-7xl font-bold mb-6 bg-gradient-to-r from-white via-blue-100 to-blue-200 bg-clip-text text-transparent">
      Design Architecture<br />with AI Precision
    </h1>
    <p className="text-xl text-slate-400 max-w-2xl mx-auto mb-10">
      Transform your architectural vision into photorealistic designs in seconds.
      From concept to visualization, powered by cutting-edge AI.
    </p>
    <div className="flex flex-col sm:flex-row gap-4 justify-center">
      <Link href="/auth/register/" className="bg-blue-600 hover:bg-blue-700 text-white px-8 py-4 rounded-xl font-semibold flex items-center justify-center gap-2">
        Start Creating Free <ArrowRight className="h-5 w-5" />
      </Link>
      <Link href="/pricing/" className="bg-slate-800 hover:bg-slate-700 px-8 py-4 rounded-xl font-semibold border border-slate-700">
        View Pricing
      </Link>
    </div>

    <div className="mt-20 grid grid-cols-2 md:grid-cols-4 gap-8 max-w-3xl mx-auto">
      <Stat number="10K+" label="Designs Generated" />
      <Stat number="2,500+" label="Active Architects" />
      <Stat number="99.9%" label="Uptime" />
      <Stat number="4.9/5" label="User Rating" />
    </div>
  </div>
</section>
```

```
      <section className="py-20 px-4 bg-slate-900/50">
        <div className="max-w-7xl mx-auto">
          <h2 className="text-3xl font-bold text-center mb-12">Why Architects Choose ArchiEngine</h2>
          <div className="grid md:grid-cols-3 gap-8">
            <FeatureCard
              icon={<Zap className="h-8 w-8 text-yellow-400" />}
              title="Lightning Fast"
              description="Generate professional architectural visualizations in under 30 seconds."
            />
            <FeatureCard
              icon={<Sparkles className="h-8 w-8 text-purple-400" />}
              title="AI-Powered Design"
              description="Describe your vision in natural language. Our AI understands architectural styles."
            />
            <FeatureCard
              icon={<Shield className="h-8 w-8 text-emerald-400" />}
              title="Commercial Ready"
              description="Full commercial usage rights on all paid plans."
            />
          </div>
        </div>
      </section>
    </div>
  );
}

function Stat({ number, label }: { number: string; label: string }) {
  return (
    <div className="text-center">
      <div className="text-3xl font-bold text-white mb-1">{number}</div>
      <div className="text-sm text-slate-400">{label}</div>
    </div>
  );
}

function FeatureCard({ icon, title, description }: { icon: React.ReactNode; title: string; description: string }) {
  return (
    <div className="bg-slate-800/50 border border-slate-700 rounded-2xl p-8 hover:border-blue-500/50
transition">
      <div className="mb-4">{icon}</div>
      <h3 className="text-xl font-bold mb-3 text-white">{title}</h3>
      <p className="text-slate-400 leading-relaxed">{description}</p>
    </div>
  );
}

====================================================================

FILE 24: app/pricing/page.tsx
-----------------------------
import { getServerSession } from 'next-auth';
import { authOptions } from '@/lib/auth';
import { PRICING_PLANS } from '@/lib/stripe';
import { PricingCard } from '@/components/pricing-card';
```

```
export default async function PricingPage() {
  const session = await getServerSession(authOptions);

  return (
    <div className="min-h-screen bg-slate-950 py-20 px-4">
      <div className="max-w-7xl mx-auto">
        <div className="text-center mb-16">
          <h1 className="text-4xl md:text-5xl font-bold text-white mb-4">Simple, Transparent Pricing</h1>
          <p className="text-xl text-slate-400">Choose the perfect plan for your architectural needs.</p>
        </div>

        <div className="grid md:grid-cols-3 gap-8 max-w-6xl mx-auto">
          <PricingCard plan={PRICING_PLANS.FREE} session={session} />
          <PricingCard plan={PRICING_PLANS.PRO} session={session} popular />
          <PricingCard plan={PRICING_PLANS.ENTERPRISE} session={session} />
        </div>
      </div>
    </div>
  );
}

====================================================================

FILE 25: app/auth/login/page.tsx
--------------------------------
'use client';

import { useState } from 'react';
import { signIn } from 'next-auth/react';
import { useRouter } from 'next/navigation';
import Link from 'next/link';

export default function LoginPage() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
  const router = useRouter();

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);
    setError('');

    const result = await signIn('credentials', {
      email,
      password,
      redirect: false,
    });
```

```
    if (result?.error) {
      setError('Invalid credentials');
      setLoading(false);
    } else {
      router.push('/dashboard/');
      router.refresh();
    }
  };

  return (
    <div className="min-h-screen bg-slate-950 flex items-center justify-center px-4">
      <div className="max-w-md w-full bg-slate-900 border border-slate-800 rounded-2xl p-8">
        <h1 className="text-3xl font-bold text-white mb-2">Welcome back</h1>
        <p className="text-slate-400 mb-6">Sign in to your ArchiEngine account</p>

        {error && (
          <div className="bg-red-500/10 border border-red-500/50 text-red-400 px-4 py-3 rounded-lg mb-4">
            {error}
          </div>
        )}

        <form onSubmit={handleSubmit} className="space-y-4">
          <div>
            <label className="block text-sm font-medium text-slate-300 mb-2">Email</label>
            <input
              type="email"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              className="w-full bg-slate-800 border border-slate-700 rounded-lg px-4 py-3 text-white focus:ring-2
focus:ring-blue-500"
              required
            />
          </div>

          <div>
            <label className="block text-sm font-medium text-slate-300 mb-2">Password</label>
            <input
              type="password"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              className="w-full bg-slate-800 border border-slate-700 rounded-lg px-4 py-3 text-white focus:ring-2
focus:ring-blue-500"
              required
            />
          </div>

          <button
            type="submit"
            disabled={loading}
            className="w-full bg-blue-600 hover:bg-blue-700 text-white font-semibold py-3 rounded-lg transition"
          >
            {loading ? 'Signing in...' : 'Sign In'}
          </button>
        </form>
```

```
        <p className="mt-6 text-center text-slate-400">
          Don't have an account?{' '}
          <Link href="/auth/register/" className="text-blue-400 hover:text-blue-300">
            Sign up
          </Link>
        </p>
      </div>
    </div>
  );
}
```

==========

FILE 26: app/auth/register/page.tsx

```
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';
import Link from 'next/link';

export default function RegisterPage() {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    password: '',
    confirmPassword: '',
  });
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
  const router = useRouter();

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);
    setError('');

    if (formData.password !== formData.confirmPassword) {
      setError('Passwords do not match');
      setLoading(false);
      return;
    }

    if (formData.password.length < 12) {
      setError('Password must be at least 12 characters');
      setLoading(false);
      return;
    }
```

```
  try {
    const res = await fetch('/api/auth/register/', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        name: formData.name,
        email: formData.email,
        password: formData.password,
      }),
    });

    const data = await res.json();
    if (!res.ok) throw new Error(data.message || 'Registration failed');
    router.push('/auth/login/');
  } catch (err: any) {
    setError(err.message);
  } finally {
    setLoading(false);
  }
};

return (
  <div className="min-h-screen bg-slate-950 flex items-center justify-center px-4">
    <div className="max-w-md w-full bg-slate-900 border border-slate-800 rounded-2xl p-8">
      <h1 className="text-3xl font-bold text-white mb-2">Create account</h1>
      <p className="text-slate-400 mb-6">Start generating AI architectural designs</p>

      {error && (
        <div className="bg-red-500/10 border border-red-500/50 text-red-400 px-4 py-3 rounded-lg mb-4">
          {error}
        </div>
      )}

      <form onSubmit={handleSubmit} className="space-y-4">
        <div>
          <label className="block text-sm font-medium text-slate-300 mb-2">Full Name</label>
          <input
            type="text"
            value={formData.name}
            onChange={(e) => setFormData({...formData, name: e.target.value})}
            className="w-full bg-slate-800 border border-slate-700 rounded-lg px-4 py-3 text-white"
            required
          />
        </div>

        <div>
          <label className="block text-sm font-medium text-slate-300 mb-2">Email</label>
          <input
            type="email"
            value={formData.email}
            onChange={(e) => setFormData({...formData, email: e.target.value})}
            className="w-full bg-slate-800 border border-slate-700 rounded-lg px-4 py-3 text-white"
            required
          />
        </div>
```

```
        <div>
          <label className="block text-sm font-medium text-slate-300 mb-2">Password (min 12 chars)</label>
          <input
            type="password"
            value={formData.password}
            onChange={(e) => setFormData({...formData, password: e.target.value})}
            className="w-full bg-slate-800 border border-slate-700 rounded-lg px-4 py-3 text-white"
            required
          />
        </div>

        <div>
          <label className="block text-sm font-medium text-slate-300 mb-2">Confirm Password</label>
          <input
            type="password"
            value={formData.confirmPassword}
            onChange={(e) => setFormData({...formData, confirmPassword: e.target.value})}
            className="w-full bg-slate-800 border border-slate-700 rounded-lg px-4 py-3 text-white"
            required
          />
        </div>

        <button
          type="submit"
          disabled={loading}
          className="w-full bg-blue-600 hover:bg-blue-700 text-white font-semibold py-3 rounded-lg transition"
        >
          {loading ? 'Creating account...' : 'Create Account'}
        </button>
      </form>

      <p className="mt-6 text-center text-slate-400">
        Already have an account?{' '}
        <Link href="/auth/login/" className="text-blue-400 hover:text-blue-300">
          Sign in
        </Link>
      </p>
    </div>
  </div>
);
}

====================================================================

FILE 27: app/dashboard/page.tsx
--------------------------------
import { getServerSession } from 'next-auth';
import { authOptions } from '@/lib/auth';
import { redirect } from 'next/navigation';
import { prisma } from '@/lib/prisma';
import Link from 'next/link';
import { Plus, Building2, Image as ImageIcon } from 'lucide-react';

export default async function DashboardPage() {
  const session = await getServerSession(authOptions);
  if (!session?.user) redirect('/auth/login/');
```

```
  const projects = await prisma.project.findMany({
    where: { userId: session.user.id },
    include: { _count: { select: { aiDesigns: true } } },
    orderBy: { createdAt: 'desc' },
  });

  const subscription = await prisma.subscription.findUnique({
    where: { userId: session.user.id },
  });

  return (
    <div className="min-h-screen bg-slate-950">
      <nav className="bg-slate-900 border-b border-slate-800 px-4 py-4">
        <div className="max-w-7xl mx-auto flex justify-between items-center">
          <div className="flex items-center gap-2">
            <Building2 className="h-6 w-6 text-blue-500" />
            <span className="text-xl font-bold text-white">ArchiEngine</span>
          </div>
          <div className="flex items-center gap-4">
            <span className="text-slate-400">Welcome, {session.user.name || session.user.email}</span>
            <Link href="/api/auth/signout" className="text-slate-400 hover:text-white">Sign Out</Link>
          </div>
        </div>
      </nav>

      <main className="max-w-7xl mx-auto px-4 py-8">
        <div className="flex justify-between items-center mb-8">
          <div>
            <h1 className="text-3xl font-bold text-white">Your Projects</h1>
            <p className="text-slate-400 mt-1">
              Plan: <span className="text-blue-400 font-semibold">{subscription?.plan || 'FREE'}</span>
            </p>
          </div>
          <Link href="/dashboard/projects/new/" className="bg-blue-600 hover:bg-blue-700 text-white px-4 py-2
rounded-lg flex items-center gap-2">
            <Plus className="h-5 w-5" /> New Project
          </Link>
        </div>
```

```
      {projects.length === 0 ? (
        <div className="text-center py-20 bg-slate-900/50 rounded-2xl border border-slate-800">
          <ImageIcon className="h-16 w-16 text-slate-600 mx-auto mb-4" />
          <h3 className="text-xl font-semibold text-white mb-2">No projects yet</h3>
          <Link href="/dashboard/projects/new/" className="bg-blue-600 hover:bg-blue-700 text-white px-6 py-3
rounded-lg inline-block">
            Create Project
          </Link>
        </div>
      ) : (
        <div className="grid md:grid-cols-2 lg:grid-cols-3 gap-6">
          {projects.map((project) => (
            <Link key={project.id} href={`/dashboard/projects/${project.id}/`} className="bg-slate-900 border
border-slate-800 rounded-xl p-6 hover:border-blue-500/50 transition">
              <div className="flex items-start justify-between mb-4">
                <div className="bg-blue-500/10 p-3 rounded-lg">
                  <Building2 className="h-6 w-6 text-blue-400" />
                </div>
                <span className="text-xs text-slate-500 bg-slate-800 px-2 py-1 rounded">{project.type}</span>
              </div>
              <h3 className="text-lg font-semibold text-white mb-2">{project.title}</h3>
              <p className="text-slate-400 text-sm mb-4">{project.description || 'No description'}</p>
              <div className="flex items-center gap-4 text-sm text-slate-500">
                <span className="flex items-center gap-1"><ImageIcon className="h-4 w-4" />
{project._count.aiDesigns} designs</span>
                <span>{new Date(project.createdAt).toLocaleDateString()}</span>
              </div>
            </Link>
          ))}
        </div>
      )}
    </main>
  </div>
  );
}

==================================================================

FILE 28: app/admin/page.tsx
---------------------------
import { getServerSession } from 'next-auth';
import { authOptions } from '@/lib/auth';
import { redirect } from 'next/navigation';
import { prisma } from '@/lib/prisma';
import { formatCurrency } from '@/lib/utils';
import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card';
import { DollarSign, Users, Zap, TrendingUp } from 'lucide-react';

export default async function AdminDashboard() {
  const session = await getServerSession(authOptions);
  if (session?.user?.role !== 'ADMIN') redirect('/unauthorized/');
```

```
  const [totalUsers, totalRevenue, activeSubscriptions, aiUsageToday] = await Promise.all([
    prisma.user.count(),
    prisma.payment.aggregate({ where: { status: 'COMPLETED' }, _sum: { amount: true } }),
    prisma.subscription.count({ where: { status: 'ACTIVE' } }),
    prisma.aiUsage.count({ where: { createdAt: { gte: new Date(new Date().setHours(0, 0, 0, 0)) } } }),
  ]);

  return (
    <div className="min-h-screen bg-slate-950 p-8">
      <h1 className="text-3xl font-bold text-white mb-8">Admin Dashboard</h1>

      <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-4">
        <Card className="bg-slate-900 border-slate-800">
          <CardHeader className="flex flex-row items-center justify-between pb-2">
            <CardTitle className="text-sm font-medium text-slate-200">Total Revenue</CardTitle>
            <DollarSign className="h-4 w-4 text-emerald-400" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-white">{formatCurrency(Number(totalRevenue._sum.amount ||
0))}</div>
          </CardContent>
        </Card>

        <Card className="bg-slate-900 border-slate-800">
          <CardHeader className="flex flex-row items-center justify-between pb-2">
            <CardTitle className="text-sm font-medium text-slate-200">Total Users</CardTitle>
            <Users className="h-4 w-4 text-blue-400" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-white">{totalUsers}</div>
          </CardContent>
        </Card>

        <Card className="bg-slate-900 border-slate-800">
          <CardHeader className="flex flex-row items-center justify-between pb-2">
            <CardTitle className="text-sm font-medium text-slate-200">Active Subscriptions</CardTitle>
            <TrendingUp className="h-4 w-4 text-purple-400" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-white">{activeSubscriptions}</div>
          </CardContent>
        </Card>

        <Card className="bg-slate-900 border-slate-800">
          <CardHeader className="flex flex-row items-center justify-between pb-2">
            <CardTitle className="text-sm font-medium text-slate-200">AI Usage Today</CardTitle>
            <Zap className="h-4 w-4 text-yellow-400" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-white">{aiUsageToday}</div>
          </CardContent>
        </Card>
      </div>
    </div>
  );
}
```

==========

FILE 29: app/api/auth/[...nextauth]/route.ts

```
import NextAuth from 'next-auth';
import { authOptions } from '@/lib/auth';

const handler = NextAuth(authOptions);
export { handler as GET, handler as POST };
```

==========

FILE 30: app/api/auth/register/route.ts

```
import { NextResponse } from 'next/server';
import { prisma } from '@/lib/prisma';
import { hashPassword } from '@/lib/password';
import { z } from 'zod';

const registerSchema = z.object({
  name: z.string().min(2),
  email: z.string().email(),
  password: z.string().min(12),
});

export async function POST(req: Request) {
  try {
    const body = await req.json();
    const { name, email, password } = registerSchema.parse(body);

    const existingUser = await prisma.user.findUnique({ where: { email } });
    if (existingUser) {
      return NextResponse.json({ message: 'User already exists' }, { status: 400 });
    }

    const hashedPassword = await hashPassword(password);

    await prisma.user.create({
      data: {
        name,
        email,
        password: hashedPassword,
        role: 'USER',
        subscription: {
          create: { plan: 'FREE', status: 'ACTIVE' },
        },
      },
    });

    return NextResponse.json({ message: 'User created successfully' }, { status: 201 });
  } catch (error) {
    return NextResponse.json({ message: 'Registration failed' }, { status: 500 });
  }
}
```

==========

FILE 31: app/api/stripe/checkout/route.ts

```
import { NextResponse } from 'next/server';
import { getServerSession } from 'next-auth';
import { authOptions } from '@/lib/auth';
import { stripe } from '@/lib/stripe';
import { prisma } from '@/lib/prisma';

export async function POST(req: Request) {
  try {
    const session = await getServerSession(authOptions);
    if (!session?.user) {
      return new NextResponse('Unauthorized', { status: 401 });
    }

    const { priceId, planType } = await req.json();
    if (!priceId) {
      return new NextResponse('Price ID is required', { status: 400 });
    }

    let user = await prisma.user.findUnique({
      where: { id: session.user.id },
      include: { subscription: true },
    });

    let customerId = user?.subscription?.subscriptionId;

    if (!customerId) {
      const customer = await stripe.customers.create({
        email: session.user.email!,
        name: session.user.name || undefined,
        metadata: { userId: session.user.id },
      });
      customerId = customer.id;
    }

    const checkoutSession = await stripe.checkout.sessions.create({
      customer: customerId,
      line_items: [{ price: priceId, quantity: 1 }],
      mode: 'subscription',
      success_url: `${process.env.NEXTAUTH_URL}/dashboard/billing/?success=true`,
      cancel_url: `${process.env.NEXTAUTH_URL}/pricing/?canceled=true`,
      subscription_data: {
        metadata: { userId: session.user.id, planType },
      },
      metadata: { userId: session.user.id, planType },
    });

    return NextResponse.json({ url: checkoutSession.url });
  } catch (error) {
    console.error('Stripe checkout error:', error);
    return new NextResponse('Internal error', { status: 500 });
  }
}
```

==========

FILE 32: app/api/stripe/webhook/route.ts

```
import { NextResponse } from 'next/server';
import { headers } from 'next/headers';
import { stripe } from '@/lib/stripe';
import { prisma } from '@/lib/prisma';
import Stripe from 'stripe';

const webhookSecret = process.env.STRIPE_WEBHOOK_SECRET!;

export async function POST(req: Request) {
  const payload = await req.text();
  const signature = headers().get('stripe-signature');

  if (!signature) {
    return new NextResponse('No signature', { status: 400 });
  }

  let event: Stripe.Event;

  try {
    event = stripe.webhooks.constructEvent(payload, signature, webhookSecret);
  } catch (err: any) {
    return new NextResponse(`Webhook Error: ${err.message}`, { status: 400 });
  }

  try {
    switch (event.type) {
      case 'checkout.session.completed': {
        const session = event.data.object as Stripe.Checkout.Session;
        const { userId, planType } = session.metadata as { userId: string; planType: string };

        const subscription = await stripe.subscriptions.retrieve(session.subscription as string);

        await prisma.subscription.upsert({
          where: { userId },
          create: {
            userId,
            status: 'ACTIVE',
            plan: planType as any,
            priceId: session.line_items?.data[0]?.price?.id,
            subscriptionId: session.subscription as string,
            currentPeriodStart: new Date(subscription.current_period_start * 1000),
            currentPeriodEnd: new Date(subscription.current_period_end * 1000),
          },
          update: {
            status: 'ACTIVE',
            plan: planType as any,
            priceId: session.line_items?.data[0]?.price?.id,
            subscriptionId: session.subscription as string,
            currentPeriodStart: new Date(subscription.current_period_start * 1000),
            currentPeriodEnd: new Date(subscription.current_period_end * 1000),
          },
        });
```

```
      await prisma.user.update({
        where: { id: userId },
        data: { role: planType === 'ENTERPRISE' ? 'PREMIUM' : 'USER' },
      });

      await prisma.payment.create({
        data: {
          userId,
          amount: session.amount_total ? session.amount_total / 100 : 0,
          currency: session.currency?.toUpperCase() || 'USD',
          status: 'COMPLETED',
          paymentIntentId: session.payment_intent as string,
          description: `Subscription to ${planType} plan`,
        },
      });
      break;
    }

    case 'customer.subscription.deleted': {
      const subscription = event.data.object as Stripe.Subscription;
      const userId = subscription.metadata.userId;

      await prisma.subscription.update({
        where: { userId },
        data: { status: 'CANCELED', plan: 'FREE' },
      });

      await prisma.user.update({
        where: { id: userId },
        data: { role: 'USER' },
      });
      break;
    }
  }

  return NextResponse.json({ received: true });
 } catch (error) {
  return new NextResponse('Webhook handling failed', { status: 500 });
 }
}
```

==========

FILE 33: app/api/ai/generate/route.ts

```
import { NextResponse } from 'next/server';
import { getServerSession } from 'next-auth';
import { authOptions } from '@/lib/auth';
import { generateArchitecturalDesign } from '@/lib/ai';
import { prisma } from '@/lib/prisma';
```

```
export async function POST(req: Request) {
  try {
    const session = await getServerSession(authOptions);
    if (!session?.user) {
      return new NextResponse('Unauthorized', { status: 401 });
    }

    const body = await req.json();
    const { prompt, projectId, style, setting, timeOfDay } = body;

    const project = await prisma.project.findFirst({
      where: { id: projectId, userId: session.user.id },
    });

    if (!project) {
      return new NextResponse('Project not found', { status: 404 });
    }

    const subscription = await prisma.subscription.findUnique({
      where: { userId: session.user.id },
    });

    const plan = subscription?.plan || 'FREE';

    const result = await generateArchitecturalDesign({
      prompt,
      userId: session.user.id,
      plan,
      projectId,
      style,
      setting,
      timeOfDay,
    });

    return NextResponse.json(result);
  } catch (error: any) {
    if (error.message.includes('limit reached')) {
      return NextResponse.json(
        { error: error.message, code: 'RATE_LIMIT' },
        { status: 429 }
      );
    }
    return NextResponse.json(
      { error: error.message || 'Generation failed' },
      { status: 500 }
    );
  }
}
```

==========

END OF FILE LIST
==========
Total: 33 files to create

FOLDER STRUCTURE:
==========

```
archiengine-pro/
├── app/
│   ├── api/
│   │   ├── auth/
│   │   │   ├── [...nextauth]/
│   │   │   │   └── route.ts
│   │   │   └── register/
│   │   │       └── route.ts
│   │   ├── stripe/
│   │   │   ├── checkout/
│   │   │   │   └── route.ts
│   │   │   └── webhook/
│   │   │       └── route.ts
│   │   └── ai/
│   │       └── generate/
│   │           └── route.ts
│   ├── auth/
│   │   ├── login/
│   │   │   └── page.tsx
│   │   └── register/
│   │       └── page.tsx
│   ├── admin/
│   │   └── page.tsx
│   ├── dashboard/
│   │   └── page.tsx
│   ├── globals.css
│   ├── layout.tsx
│   ├── page.tsx
│   └── pricing/
│       └── page.tsx
├── components/
│   ├── providers.tsx
│   ├── pricing-card.tsx
│   └── ui/
│       └── card.tsx
├── lib/
│   ├── prisma.ts
│   ├── auth.ts
│   ├── password.ts
│   ├── stripe.ts
│   ├── ai.ts
│   └── utils.ts
├── prisma/
│   └── schema.prisma
├── types/
│   └── next-auth.d.ts
├── .env.local
├── .env.local.example
├── .gitignore
├── README.md
├── next.config.js
├── package.json
├── postcss.config.js
└── tailwind.config.ts
```